

Testing Automotive Reactive Systems using Timed Automata

Jan Sobotka¹, Jiří Novák¹

¹ Czech Technical University in Prague, Faculty of Electrical Engineering,
Technická 2, 166 27 Prague, Czech Republic
jan.sobotka@fel.cvut.cz, www.fel.cvut.cz

Abstract— Model oriented testing methods are known relatively long time, but the right time for viable deployment is coming up now with the continuously growing amount of vehicle electronic systems. Paper proposes a new Model-Based Testing approach to testing of automotive electronics systems. The key concept is the utilization of Timed Automata for online testing of automotive electronic control units. System modeling, the new tool, a bit of necessary technology and preliminary results intended to identify future research goal are presented.

Keywords— *Model-Based, Testing, Timed; Automaton; Automotive; ECU*

I. INTRODUCTION

Since the massive expansion of Electronic Fuel Injection systems in eighty's, the complexity of automotive electronic systems is growing. The growth is increased by utilization of new functionalities towards to fully autonomous vehicles. Examples of these new features are driving assistance technologies and connected car solutions. A number of electronics combined with pressure on price, short development time and high-quality standards create perfect area for new testing methods deployment.

Paper deals with the utilization of Model-Bases Testing principles to the area of automotive integration testing. It is high-level function oriented testing of almost finished car electronics system or part of that system. A function can be implemented in a single electronic control unit (ECU) or distributed over multiple ECUs. Detailed description of integration testing process can be found in our previous conference paper [1]. Example of such kind of system is provided in the paper.

Every important part of proposed testing solution is presented. The testing concept is depicted in the Fig. 1. Start with necessary system modeling using Time Automata. Used time automaton system definition is stated.

It continues with describe newly developed testing tool. And finishes with necessary technology aspects enables

connection to real system under test (SUT). Altogether it enables Hardware-in-the-Loop testing automotive electronics systems driven by timed automata model. Also it creates platform for future research outlined in section IX.

II. RELATED WORK

Timed automata are proved as very suitable for handling Real-Time systems properties. Therefore they are frequently used as the underlying formalism for various software tools. Mostly the tools deal with verification and validation. Overview of timed automata related tools is included in the paper [2]. Scope of the paper [2] is primary to provide survey of timed automata variants. Utilization of timed automata in testing tools is not so widespread as for mentioned verification and validation. Closely related to presented work are tools CoVer [3] and TRON [4]. Both belong to UPPAAL family. Common feature is employment of UPPAAL model checking engine verifyta for test generation. CoVer introduces the interesting solution for coverage criteria description named Observer Automata. Test generation is done offline in opposite the TRON tool which uses online approach. Presented approach is inspired previously mentioned works, but with the bigger focus on testing itself. Research in meaningful test selection criteria and test control algorithms suitable for targeted area is the main objective.

III. TIMED AUTOMATON

Theory of timed automata was originally published by Alur and Dill [5]. The term timed automata usually means slightly modified version called Timed Safety Automata [6] which uses local invariant conditions to ensure automaton progress. Informally, the timed automaton is an oriented graph which contains states and transitions. One of the states is called initial. Transitions are labeled by guards which enable relevant transition. One of the key concepts of timed automata theory is parallel composition of individual automata to the network of timed automata. The theoretical background for this operation provides parallel composition operator know from communicating sequential processes (CSP).

¹ This work was supported by the Grant Agency of the Czech Technical University in Prague, project Model-Based Testing methods for automotive electronics systems (grant No. SGS16/171/OHK3/2T/13) and by the support of EU Regional Development Fund in OP R&D for Innovations (OP VaVpI) and The Ministry of Education, Youth and Sports, Czech Republic, project # CZ.1.05/2.1.00/03.0125 Acquisition of Technology for Vehicle Center of Sustainable Mobility.

Formal definition of a single timed automaton, given by [7], is:

- A timed automaton A is a tuple $A = (N, l_0, E, I)$ where
- N is a finite set locations (or nodes),
- $l_0 \in N$ is initial location,
- $E \subseteq N \times B(C) \times \Sigma \times 2^C \times N$ is the set of edges and
- $I : N \rightarrow B(C)$ assigns invariants to locations
- We shall write $l \xrightarrow{g,a,r} l'$ when $\langle l, g, a, r, l' \rangle \in E$

Local invariant is constraint in form $x \leq n, x < n$ where n is natural number.

Timed Safely Automata are implemented in UPPAAL [8] tool, which is used as modelling environment for proposed concept. Example of timed automata is show in the pics 4, 5, 6. Deeper explanation of the theory is provided e.g. in [7].

IV. TESTING CONCEPT

Testing process utilizing presented tool Taster is based on following scheme. Fundamentally, it is Online Model-Based Testing (MBT) using Hardware-in-the-Loop method. Tested system and its environment are modelled by timed automata network in UPPAAL tool. Developed model is afterwards loaded by Taster. The oracle function is provided directly by a SUT model. Test is driven by environment model, and its exploration produces testing stimuli. Connection to tested hardware is provided by adapter between our tool and NI VeriStand. Described workflow is depicted in the Fig. 1.

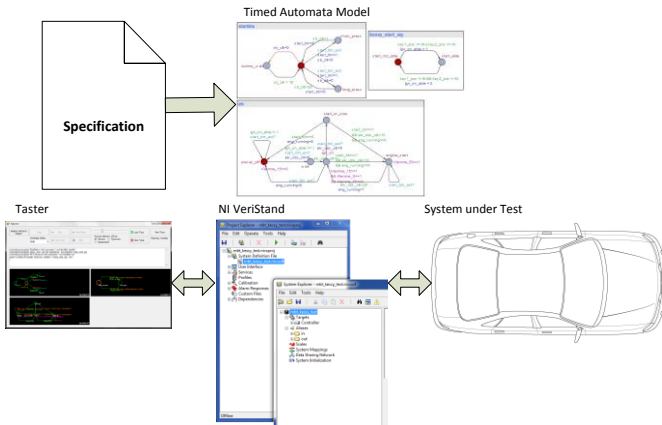


Figure 1. Workflow of proposed testing solution

V. SYSTEM DESCRIPTION

As mentioned in previous sections, SUT and its environment are described by timed safely automata using UPPAAL tool editor. On tested system class, original modeling capabilities were restricted to reasonable subset. Supported data types are *bool*, *int*, *clock* and synchronization type *chan*. Edges can contain *guard*, *sync* and *update* expressions. Besides time automata modelling language implemented by

UPPAAL tool, Taster additionally utilizes labeling of timed automata states by *relevance*. *Relevance* is natural number assigned to an automaton state. This parameter is used to distinguish the parts of a SUT with different criticality and thus influences the model exploration. Example of timed automaton with assigned relevance number is in the Fig. 5.

VI. TASTER

Presented concept is implemented in testing software tool called Taster. The tool provides platform for implementation and evaluation of proposed research ideas. First version of this tool was implemented by [9]. Software architecture can be divided into two parts and is depicted in Fig. 2. First part – the model analyzer provides lexical and syntactical analysis, parses the input model from UPPAAL 4 format and stores it in object model suitable for online exploration. Second part is responsible for testing itself. It performs model execution, serves stimuli by controlling SUT inputs and observes responses by reading SUT outputs. Individual test case is recorded by trace logger enables further analysis or replay function.

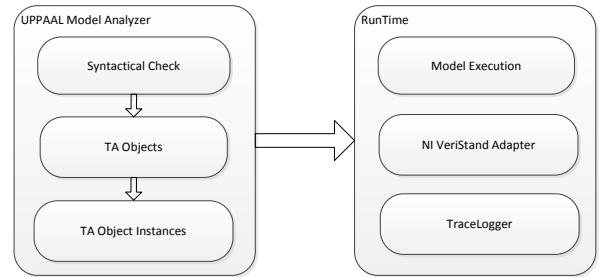


Figure 2. Taster tool architecture overview

In a nutshell, Taster offers following features. System model viewer enables inspection of loaded model. Test adaptor is providing the connection with SUT. The adaptor connects Taster to National Instrument VeriStand Real-Time testing software. System testing is possible using three different exploration strategies. The random strategy is suitable for black box testing of systems without any additional knowledge about SUT. Systematic strategy approximates conformance testing. Moreover, finally, algorithm labeled experimental uses relevance information described in section V. States with higher relevance number are favorited against lower relevant states or states without relevance assigned. A test run is terminated in one of the following ways. Invariant condition is violated, test is terminated by user, or a selected coverage criterion is fulfilled. Printscreen of runtime windows is depicted in Fig. 3.

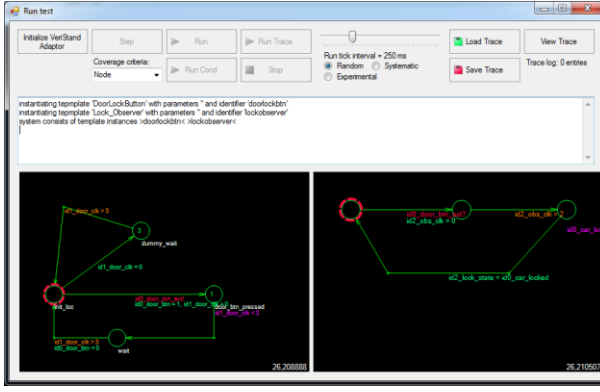


Figure 3. Taster Run Screen

VII. TESTING TECHNOLOGY/TECHNICAL DETAILS

This paragraph lists the most important technical or implementation details of presented MBT testing solution. Reader interested in deployment of MBT method to real word problems might find these hints helpful. Taster tool is developed in C# using .NET technology. Syntactical analyzer is generated by language recognition software ANTLR. Expression evaluation necessary for model execution is solved by Shunting-yard algorithm [10]. System models, as well as the test run traces, are stored in XML-formatted file. Hardware-in-the-Loop testing method is realized on National Instrument platform with NI VeriStand software environment. Taster is connected to that platform by .NET API provided by National Instruments.

VIII. KESSY SYSTEM TESTING

Following example shows application of presented concept on testing of a keyless access system. As the name suggest, purpose of keyless access system is to allow vehicle entry and engine start without using a key (only key RFID tag have to be detected). The example system function (textual specification) is captured by this description. Door locking system is controlled by lock button built in the driver side door handle. Engine operation is controlled by start/stop button. Ability to lock and unlock the door, as well as start or stop the engine, is determined by detected key position. System contains two equal keys RFID tags marked Key 1 and Key 2. System recognizes following states of each key: detected outside the car, detected inside the car and not detected. The door unlocking is not possible, if no key is detected outside the car. The door locking is not allowed if a key is detected inside the car. The engine start requires a key to be detected inside a car.

Conversion of textual specification into timed automata model results into following. According to the paper format, it is not possible to include complete model and thus only the most important parts are depicted.

The SUT and its environment are modelled by individual automata. The environment is covered by start button (Fig. 4), door lock button (Fig. 5) and key position. These models are compromise between fully permissive model and fully restrictive model. It simulates meaningful buttons operations,

but not meaningful behavior of imaginary driver. Also idling, which is very natural for system like this, is modeled.

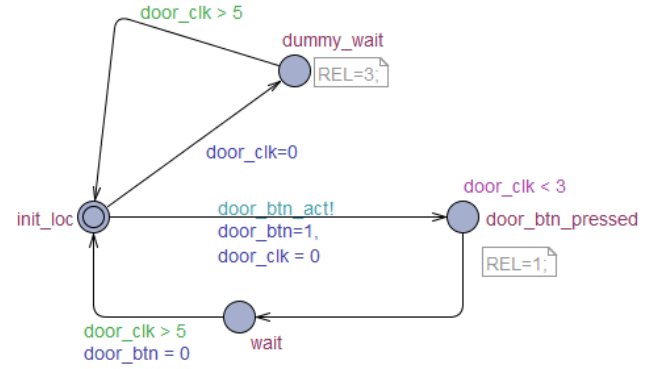


Figure 4. Door Button Model

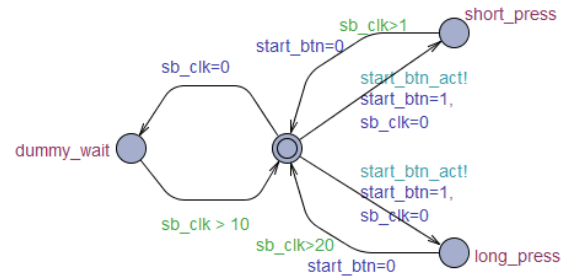


Figure 5. Start Button Model

The SUT model is represented by engine state observer depicted in Fig. 6. System response to short and long start button press, ability for the engine start and the engine actual state are captured. Also model presents how multi-level approach can be combined. High level approach need to differ between engine state (stop or running) only. Low level model extension for power supply status helps to localize bugs in the case of error detection. Power supply wires are labeled with prefix Klemme originating in German language and it is common in automotive industry. SUT is implemented in LabView as NI VeriStand Simulation Model. This allows efficient modification of tested system required for the tool development.

IX. RESEARCH PROPOSAL

Development of new MBT testing tool itself is not major objective. Instead, the Taster should provide a platform for experimental evaluation of our research ideas. Some of them are outlined in this section. Demands on testing quality are growing with increasing electronics systems complexity. Therefore, this research is focused on specialized Model-Based testing methods suitable for automotive integration testing process.

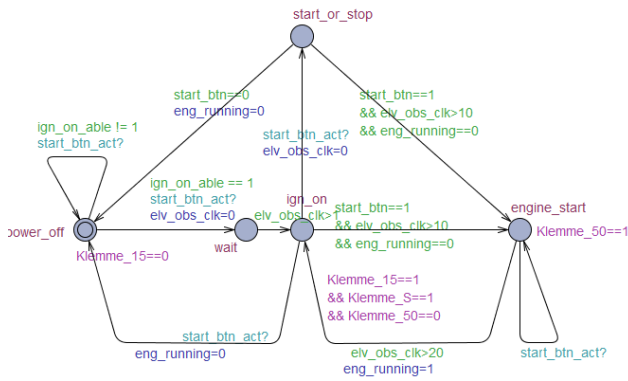


Figure 6. Engine State Observer

These methods are not intended to substitute currently used methods required by various road regulations, but to offer attractive way how to decrease number of unrevealed problems in a SUT.

First idea already proposed in this paper is to extend timed automata models by *relevance* numbers. This number indicates criticality of the SUT parts. It can be used to state space exploration optimization. *Relevance* origin is considered in expert knowledge. It can be inscribed by model developer based on SUT knowledge or machine extracted from test results database. Automatic *relevance* extraction is topic of future research.

Situation covered in test suites are predominantly designed from driver's point of view (i.e. are use case oriented). Uncommon system usage is frequently not covered by test suite at all. Hypothesis is that generation of unconventional test step sequences might reveal some extra bugs. Unconventional does not mean random; it means situations not conforming to a typical system use case diagram. Objective is generation of test runs simulating anomalous driver behavior (e.g. remarkable quick or slow interact with a system).

One of the test objectives in targeted area (integration testing of automotive electronics) is to investigate, if individual parts of the SUT are able to work together in synergy without any undesirable interference. Similarly to idea with imitating unusual car-driver interaction, hidden relation between parts of distributed systems could be attractive test objective. Black box, as well as grey box approach, is being considered for this problem.

X. CONCLUSION

New Model-Based solution including software implementation for testing of automotive electronics systems was proposed. Targeted application is testing of comfort part of vehicle electronics systems, such as door locking, exterior/interior lighting, and air condition. However, the

application is not limited to automotive comfort systems or the vehicles itself. As a result, new software tool Taster implementing the method was presented. The suggested method utilizes well known timed automata formalism as system description language. The model has dual purpose. At first, provides the function of test oracle (defines SUT correct behavior). Environment model and its exploration generate test stimuli. Testing using Hardware-in-the-Loop principle is realized by connection to the National Instrument hardware platform by the NI VeriStand software environment. Furthermore, Taster tool creates the platform for future research in the domain of automotive integration testing. Opportunity to examine Taster on genuine new vehicle systems came from cooperation with our industrial partner Škoda Auto a. s.

ACKNOWLEDGMENT

We would like to thank former master degree student Tomáš Grus for the first implementation of the test tool and help with project in the beginning.

REFERENCES

- [1] Sobotka, J., & Novak, J. (2013). Automation of automotive integration testing process. *Proceedings of the 2013 IEEE 7th International Conference on Intelligent Data Acquisition and Advanced Computing Systems, IDAACS 2013*, 1(September), 349–352. doi:10.1109/IDAACS.2013.6662704
- [2] Waez, M. T. Bin, Dingel, J., & Rudie, K. (2013). A survey of timed automata for the development of real-time systems. *Computer Science Review*, 9, 1–26. doi:10.1016/j.cosrev.2013.05.001
- [3] Hessel, A., & Pettersson, P. (2007). CoVer-a real-time test case generation tool. In *19th IFIP International Conference on Testing of Communicating Systems and 7th International Workshop on Formal Approaches to Testing of Software*.
- [4] Larsen KG; Nielsen, B, Skou, A. (n.d.). UPPAAL TRON: testing real time systems ONLINE.
- [5] Alur, R., & Dill, D. (1990). Automata for modeling real-time systems. *Proceedings of the Seventeenth International Colloquium on Automata Languages and Programming*, 443(443), 322–335. doi:10.1007/BFb0032042
- [6] Henzinger, T. A., Nicollin, X., Sifakis, J., & Yovine, S. (1994). Symbolic model checking for real-time systems. *Information and Computation*, 111(2), 193–244. doi:10.1006/inco.1994.1045
- [7] Bengtsson, J., Bengtsson, J., Yi, W., & Yi, W. (2004). Timed automata: Semantics, algorithms and tools. *Lecture Notes in Computer Science*, 3098(316), 87–124. doi:10.1007/978-3-540-27755-2_3
- [8] Team, U., & others. (2010). UPPAAL 4.0: Small tutorial, November 2009. URL http://www.it.uu.se/research/group/darts/uppaal/small_tutorial. Pdf. Accessed: December, 10.
- [9] Tomáš Grus. *Implementation of Integration Testing Test Cases Generation Tool*. Master's Thesis, CTU in Prague. 2014.
- [10] The Shunting-Yard Algorithm <http://www.reedbeta.com/blog/2011/12/11/the-shunting-yard-algorithm/>